

FICHA TÉCNICA — CAMILA VIALI (Backend contable / conciliación de pagos)

1. Información general

Nombre del proyecto: camila-viali-backend **Tipo de aplicación:** API de integración contable y conciliación de pagos (back-office), con panel administrativo web planeado **Versión:** MVP en producción (backend) / Frontend aún no iniciado **Stack tecnológico actual (backend):**

- Python 3.10
- FastAPI
- SQLAlchemy (ORM)
- MySQL
- Pydantic
- PyJWT + bcrypt (autenticación)
- Pandas + openpyxl (generación de reportes Excel)
- Integraciones HTTP con Bsale API y API SUNAT

Stack tecnológico planeado (frontend):

- Angular
- TypeScript
- Tailwind CSS

El frontend todavía no existe. Esta ficha documenta el backend tal como está implementado hoy y define el alcance esperado del panel administrativo que se construirá sobre él.

2. Descripción del negocio

Camila Viali es una empresa retail que opera su punto de venta / ERP sobre **Bsale**. El negocio necesita:

- Centralizar en una base de datos propia (MySQL) los documentos electrónicos (boletas, facturas, notas de crédito), pagos, tipos de cambio y tipos de pago que se generan en Bsale.
- Conciliar los pagos con tarjeta contra las operaciones reportadas por **Niubiz** (pasarela de pagos), para confirmar que el dinero depositado por el operador corresponde a las ventas registradas.
- Generar reportes contables (libro de ventas, asiento contable, asiento de cancelación) en formato Excel para ser cargados al sistema contable (SIIGO).
- Mantener un sistema de usuarios internos con roles y permisos para controlar quién puede ejecutar cada proceso.

Hoy esta lógica vive únicamente como una API (FastAPI) que se opera vía llamadas HTTP/Postman o procesos batch. El objetivo de la siguiente fase es exponer esa misma lógica a través de un panel administrativo web (Angular) para que el equipo contable/financiero no dependa de llamar la API manualmente.

3. Objetivo del proyecto

1. Mantener sincronizada la base de datos local con los documentos, pagos, tipos de cambio y tipos de pago de Bsale.
 2. Cargar y conciliar automáticamente los reportes de Niubiz contra los pagos registrados.
 3. Permitir revisar y confirmar manualmente el "libro de cancelación" antes de darlo por cerrado.
 4. Generar reportes contables (libro de ventas, asiento contable) listos para el área de contabilidad.
 5. Administrar usuarios internos, roles y permisos.
 6. Construir un panel web (Angular) que permita ejecutar y monitorear todos estos procesos sin usar la API directamente.
-

4. Evento central del sistema

El evento central del sistema es:

Conciliar un pago recibido por Niubiz contra el pago registrado en Bsale, y dejar evidencia contable de esa conciliación.

Flujo principal:

```
Bsale (ERP / POS)
↓
Se generan documentos y pagos
↓
Backend sincroniza documentos, pagos, tipos de cambio (insert_*)
↓
Se sube el archivo de liquidación Niubiz (upload_niubiz)
↓
Se ejecuta el matching (matching_niubiz)
↓
Cada pago queda asociado (o no) a una operación Niubiz
↓
Se genera el asiento de cancelación para revisión (cancellation_book)
↓
Un responsable revisa el Excel generado
↓
Se confirma la conciliación (cancellation_book/confirm)
↓
Se generan libro de ventas y asiento contable (sales_book / book_entry)
↓
Contabilidad recibe los archivos listos para SIIGO
```

5. Tipos de usuario

El sistema usa un modelo de **roles y permisos dinámico** (tablas `roles`, `permissions`, `role_permission`), no roles fijos en código. Un usuario (`users`) pertenece a un rol (`role_id`), y cada rol tiene un conjunto de permisos asignados.

Esto permite crear roles como los siguientes (sugeridos para el panel, configurables sin tocar código):

5.1 Administrador

- Gestionar usuarios (crear, editar, desactivar).
- Gestionar roles y permisos.
- Acceso completo a todos los módulos de conciliación y reportes.

5.2 Contable / Financiero

- Ejecutar sincronización de documentos, pagos y tipos de cambio.
- Cargar archivos Niubiz y ejecutar el matching.
- Revisar y confirmar el libro de cancelación.
- Generar libro de ventas y asiento contable.
- No puede gestionar usuarios ni roles.

5.3 Consulta (solo lectura)

- Visualizar documentos, pagos y reportes generados.
- No puede ejecutar procesos de carga, matching ni confirmación.

Los permisos concretos (`permission`) deben definirse como catálogo en la tabla `permissions` y asignarse por rol vía `role_permission` . El backend ya valida permisos por endpoint mediante `PermissionChecker` .

6. Módulos actuales del backend

6.1 Autenticación (/users)

- `POST /users/login` — autentica por `username` o `email` + password (bcrypt), devuelve un JWT (`access_token` , expira en 90 minutos).
- `GET /users/me` — devuelve el usuario autenticado a partir del token.
- Todo el API además exige un header global `X-Token` (secreto compartido, `SECRET_TOKEN`) validado en `dependencies.verify_token` para **todas** las rutas, independientemente del JWT.
- Autorización fina por permiso vía `PermissionChecker(required_permission)` , que revisa si el rol del usuario autenticado tiene el permiso requerido.

6.2 Usuarios (/users)

- `GET /users/` — lista usuarios activos (con sus permisos de rol cargados).
- `POST /users/` — crea usuario (hash de password con bcrypt).
- `GET /users/{username}` — endpoint de prueba (pendiente de completar, hoy solo hace eco del username).

6.3 Roles (/roles)

- `GET /roles/` — lista roles.
- `POST /roles/` — crea un rol.

Falta CRUD completo de roles (editar, eliminar, asignar/quitar permisos) y CRUD de permisos — ver sección 14 (pendientes).

6.4 Documentos, pagos y reportes (/bills)

Este es el módulo principal del sistema. Endpoints actuales:

Endpoint	Método	Función
/bills/extract_data	GET	Sincroniza documentos (boletas/facturas/NC), clientes, monedas y desgloses de impuestos desde Bsale hacia la BD local, para un rango de fechas de emisión.
/bills/insert_payment_types	POST	Sincroniza el catálogo de tipos de pago desde Bsale.
/bills/insert_payments	POST	Sincroniza pagos desde Bsale para una fecha de registro (<code>recordDate</code>), incluyendo N° de operación asociado.
/bills/insert_exchanges	POST	Sincroniza tipos de cambio (compra/venta) desde SUNAT para un mes/año.
/bills/upload_niubiz	POST	Carga un archivo Excel de liquidación de Niubiz y lo inserta en <code>operaciones_niubiz</code> .
/bills/matching_niubiz	POST	Ejecuta el algoritmo de conciliación entre <code>payments</code> y <code>operaciones_niubiz</code> para un rango de fechas, en dos pasadas (por voucher/código de autorización primero, luego por tolerancia de tiempo/fecha). Persiste resultados en <code>payment_source_match</code> .
/bills/cancellation_book	GET	Genera el Excel de asiento de cancelación con los matches pendientes de confirmar (<code>canceled = 0</code>) en un rango de fechas. No modifica datos.
/bills/cancellation_book/confirm	POST	Marca como confirmados (<code>canceled = 1</code>) los <code>payment_source_match</code> cuyos ids fueron revisados manualmente en el Excel anterior.
/bills/sales_book	GET	Genera el libro de ventas (formato tributario Perú: código tributario, serie, receptor, IGV, neto, total, referencias de notas de crédito, anulaciones) para un rango de fechas.
/bills/book_entry	GET	Genera el asiento contable (formato SIIGO: cuentas, centros de costo/subcentro, referencias, impuestos desglosados) para un rango de fechas.

Reglas de negocio relevantes ya implementadas:

- El matching prioriza coincidencias exactas por voucher/código de autorización, luego por tiempo exacto, luego por tolerancia de tiempo, luego por fecha (fallback), evitando que una operación Niubiz se use dos veces.
- El proceso de matching es incremental: pagos ya conciliados y operaciones ya usadas no se vuelven a procesar dentro del mismo rango.
- El libro de ventas y el asiento contable invierten los montos en notas de crédito y anulan los importes en documentos anulados (`canceled = true`).
- Las series de documento se traducen a su equivalente SIIGO (`get_serial_number_siigo`) y se asignan a centro/subcentro de costo.

7. Integraciones externas

Integración	Uso
Bsale API (<code>ACCESS_TOKEN_BSALE</code>)	Origen de documentos, clientes, monedas, pagos, tipos de pago y desgloses de impuestos.
API SUNAT (<code>API_SUNAT_URL</code> , <code>API_SUNAT_TOKEN</code>)	Origen de tipos de cambio oficiales (compra/venta) por día.
Niubiz	No es una API en línea en el código actual: se procesa mediante carga manual de archivo Excel de liquidación (<code>upload_niubiz</code>).

8. Modelo de datos actual

users

```
id, username, email, password (hash bcrypt), first_name, last_name, status, role_id (FK roles)
```

roles / permissions / role_permission

```
roles: id, role, description, status  
permissions: id, permission, description, status  
role_permission: role_id (FK), permission_id (FK), status
```

clients

```
id, first_name, last_name, email, code, phone, company,  
company_or_person (bool), is_foreigner (bool)
```

coins

```
id, name, symbol, code
```

document_types

```
id, name, code, is_electronic_document, is_credit_note, commercial_code
```

documents

```
id, emission_date, expiration_date, generation_date, serial_number, tracking_number,  
total_amount, net_amount, tax_amount, exempt_amount,  
address, district, city, canceled, url_pdf,  
id_document_type (FK), id_client (FK), id_coin (FK)
```

breakdowns (desglose de impuestos por documento)

```
id, document_id (FK), tax_id (FK), amount
```

taxes

```
id, name, percentage, forAllProducts, amountTax, code
```

payment_types

```
id, name, isVirtual, isCheck, isCreditNote, isClientCredit, isCash, isCreditMemo, inactive, isAgreementBank
```

payments

```
id, recordDate, amount, operationNumber, isCreditPayment, createdAt, inactive,  
id_document (FK), id_payment_type (FK), id_return (FK)
```

returns (notas de crédito / devoluciones)

```
id, code, return_date, motive, type, amount,  
credit_note_id (FK documents), reference_document_id
```

operaciones_niubiz

```
id_operacion, ruc, razon_social, cod_comercio, nombre_comercial,  
fecha_y_hora_operacion, fecha_de_deposito,  
producto, tipo_de_operacion, tarjeta, origen_tarjeta, tipo_de_tarjeta, marca_de_tarjeta,  
moneda, importe_de_operacion, es_dcc, monto_dcc,  
comision_total, comision_niubiz, igv, suma_depositada, estado,  
cuenta_banco_pagador, banco_pagador, n_serie_terminal,  
codigo_de_autorizacion, n_referencia, n_lote, n_voucher, tipo_de_abono
```

payment_source_match (conciliación)

```
id, id_payment (FK payments, único), id_operacion (FK lógica a la fuente),  
source ('NIUBIZ' | ...), match_type (EXACT | TIME_TOLERANCE | WEB_ONLY | DATE_FALLBACK | VOUCHER_MATCH | AUTH  
time_diff_secs, matched_at, canceled (bool), canceled_at
```

centers / subcenters

```
centers: id, description  
subcenters: id, subcenter_code, description, id_center (FK)
```

exchanges

```
id, day, value, type ('C' compra | 'V' venta)
```

approvals

```
id, bsale, siigo -- mapea series/códigos de documento de Bsale a SIIGO
```

9. Relaciones principales

```
User
└─ belongsTo Role
    └─ belongsToMany Permission (role_permission)

Client
└─ hasMany Document

Document
├─ belongsTo DocumentType
├─ belongsTo Client
├─ belongsTo Coin
├─ hasOne Return (si es nota de crédito)
├─ hasMany Breakdown
└─ hasMany Payment

Payment
├─ belongsTo Document
├─ belongsTo PaymentType
├─ belongsTo Return (si es pago por devolución)
└─ hasOne PaymentSourceMatch

PaymentSourceMatch
└─ referencia lógica a OperacionNiubiz (por id_operacion + source)

Center
└─ hasMany Subcenter

Breakdown
├─ belongsTo Document
└─ belongsTo Tax
```

10. Estructura actual del backend (FastAPI)

```
camila-viali-backend/  
├─ main.py                # instancia FastAPI, middlewares, routers  
├─ dependencies.py        # verify_token (X-Token global)  
├─ models/  
│   ├── engine/connection.py # engine SQLAlchemy + get_db()  
│   └─ *.py                # un modelo por tabla  
├─ routers/  
│   ├── users.py           # auth, usuarios  
│   ├── roles.py           # roles  
│   └─ bills.py            # documentos, pagos, niubiz, reportes contables  
└─ schemas/  
    ├── user.py  
    ├── role.py  
    └─ bill.py              # DTOs de request/response del módulo bills
```

No existen todavía: capa de `services` separada de los routers, tests automatizados, ni `requirements.txt` con versiones fijadas (el archivo existe pero está vacío).

11. Seguridad

Implementado:

- Header `X-Token` obligatorio en toda la API (secreto compartido vía variable de entorno), aplicado globalmente como dependencia de la app.
- Autenticación por JWT (`PyJWT`), firmado con clave simétrica, expiración de 90 minutos.
- Password hasheado con `bcrypt` .
- Autorización fina por permiso (`PermissionChecker`) a nivel de endpoint.
- CORS restringido a orígenes explícitos (`localhost` , `localhost:4200` ya habilitado pensando en Angular).

Pendiente / a reforzar antes de exponer un panel web:

- Mover el secreto del JWT (`key='secret'`) a variable de entorno (`SECRET_KEY` ya existe en `.env` pero no se está usando en el código de firma).
 - Aplicar `PermissionChecker` de forma consistente en **todos** los endpoints de `bills.py` (hoy no todos lo usan).
 - Manejo de expiración/errores de JWT con respuestas controladas (hoy `jwt.decode` puede lanzar excepción no controlada).
 - Rate limiting en `/users/login` .
 - No commitear archivos generados con datos reales (Excel/CSV de reportes) — ver sección 14.
-

12. Frontend (planeado) — Angular + TypeScript + Tailwind CSS

Aún no existe código de frontend. Se construirá como una SPA en Angular que consuma esta API.

12.1 Consideraciones de integración

- El backend ya permite CORS desde `http://localhost:4200` (puerto por defecto de Angular).
- Toda petición deberá incluir el header `X-Token` (secreto de aplicación) y el `Authorization: Bearer <JWT>` una vez logueado.
- Angular deberá manejar la renovación/expiración del JWT (90 min) con un interceptor HTTP.

12.2 Estructura conceptual sugerida

```
src/app/
├── core/
│   ├── interceptors/      # auth.interceptor.ts (X-Token + Bearer), error.interceptor.ts
│   ├── guards/            # auth.guard.ts, permission.guard.ts
│   └── services/          # auth.service.ts, http-base.service.ts
├── shared/
│   ├── components/        # tablas, botones, modales, date-range-picker
│   └── pipes/ directives/
├── features/
│   ├── auth/
│   │   └── login/
│   ├── dashboard/
│   ├── users/             # CRUD usuarios
│   ├── roles/             # CRUD roles y asignación de permisos
│   ├── documents/         # consulta de documentos sincronizados
│   ├── payments/          # consulta de pagos sincronizados
│   ├── niubiz/
│   │   ├── upload/        # carga de archivo de liquidación
│   │   └── matching/       # ejecutar y revisar matching
│   ├── cancellation-book/  # revisión y confirmación del libro de cancelación
│   └── reports/
│       ├── sales-book/    # libro de ventas
│       └── book-entry/     # asiento contable
├── app.routes.ts
└── app.config.ts
```

Estilos con **Tailwind CSS** (utility-first), sin librería de componentes adicional salvo que el equipo decida incorporar una (ej. PrimeNG, Angular Material) — a definir.

12.3 Pantallas mínimas del MVP de frontend

1. **Login** — consume `POST /users/login`.
2. **Dashboard** — resumen de últimas sincronizaciones, pagos sin conciliar, pendientes de confirmación.
3. **Usuarios** — listar/crear (`/users`), pendiente: editar/desactivar.
4. **Roles y permisos** — listar/crear roles (`/roles`), pendiente: asignar permisos.
5. **Sincronización** — formularios para disparar `extract_data`, `insert_payments`, `insert_payment_types`, `insert_exchanges` por rango/fecha.
6. **Niubiz** — subir archivo (`upload_niubiz`), ejecutar matching (`matching_niubiz`), ver resultados (matcheados/sin match).

7. **Libro de cancelación** — generar (`cancellation_book`), revisar tabla en pantalla (no solo Excel), confirmar selección (`cancellation_book/confirm`).
 8. **Reportes** — generar y descargar libro de ventas y asiento contable por rango de fechas.
-

13. Responsive

Prioridad media-alta: es un panel interno de uso administrativo, principalmente en desktop, pero debe ser usable en tablet. Mobile no es prioritario para el MVP del frontend.

14. Pendientes conocidos del backend (a resolver antes o durante la construcción del frontend)

1. `requirements.txt` vacío — no hay versiones fijadas de dependencias.
 2. Archivos generados (reportes Excel/CSV con datos reales de clientes y operaciones) están sueltos en la raíz del repo y sin trackear — **no deben commitearse**; deben excluirse vía `.gitignore` y, de ser necesario, moverse a un storage fuera del repositorio.
 3. `__pycache__/*.pyc` versionados en git — deben ignorarse.
 4. CRUD incompleto de usuarios (editar, desactivar) y de roles/permisos (editar, eliminar, asignar permisos).
 5. `GET /users/{username}` es un stub sin implementar.
 6. Clave de firma JWT hardcodeada (`'secret'`) en vez de usar `SECRET_KEY` del `.env` .
 7. Sin tests automatizados.
 8. `PermissionChecker` no aplicado de forma uniforme en todos los endpoints sensibles de `bills.py` .
 9. No existe endpoint para editar/eliminar documentos, pagos o configuraciones (clients, centers, subcenters, taxes) desde la API — hoy se insertan solo vía sincronización con Bsale.
-

15. Prioridad de implementación sugerida

BLOQUE 1 — Saneamiento del backend

1. Congelar `requirements.txt` .
2. Sacar del repo y del historial los archivos de datos reales (Excel/CSV) y el `__pycache__` .
3. Mover el secreto JWT a `SECRET_KEY` de entorno.
4. Manejo de errores controlado en `get_current_user` .

BLOQUE 2 — Completar API de soporte al panel

1. CRUD completo de usuarios (editar, activar/desactivar).
2. CRUD completo de roles y gestión de permisos (asignar/quitar).
3. Endpoints de consulta paginada para documents, payments, operaciones_niubiz (hoy los reportes solo devuelven Excel/objetos completos, no hay listados paginados pensados para UI).
4. Aplicar `PermissionChecker` en todos los endpoints de `bills.py` .

BLOQUE 3 — Frontend: base del proyecto Angular

1. Crear proyecto Angular + Tailwind CSS + TypeScript estricto.
2. Layout base, routing, interceptores (`X-Token` , `Authorization`), guards.
3. Login conectado a `/users/login` + manejo de sesión.

BLOQUE 4 — Frontend: módulos administrativos

1. Usuarios y Roles (CRUD).
2. Dashboard con indicadores básicos (documentos sincronizados, pagos sin conciliar, pendientes de confirmación).

BLOQUE 5 — Frontend: flujo de conciliación (flujo crítico del negocio)

1. Pantalla de sincronización (documentos, pagos, tipos de cambio, tipos de pago).
2. Carga de archivo Niubiz + ejecución de matching + visualización de resultados.
3. Revisión y confirmación del libro de cancelación desde la UI (no solo Excel).

BLOQUE 6 — Frontend: reportes

1. Generación y descarga de libro de ventas y asiento contable por rango de fechas.
-

16. Flujo crítico que debe funcionar de punta a punta

1. Usuario inicia sesión en el panel Angular.
2. Dispara la sincronización de documentos y pagos de un rango de fechas.
3. Sube el archivo de liquidación Niubiz del mismo periodo.
4. Ejecuta el matching.
5. Ve en pantalla cuántos pagos quedaron conciliados y cuántos sin match.
6. Genera el asiento de cancelación y lo revisa en pantalla.
7. Confirma los registros revisados.
8. Genera el libro de ventas y el asiento contable del periodo.
9. Descarga ambos archivos para entregarlos a contabilidad.

Si este flujo no funciona completo desde la UI, el panel administrativo no debe considerarse terminado.

17. Testing

No existen tests hoy. Prioridad sugerida para el backend:

- ✓ login exitoso / credenciales inválidas
 - ✓ endpoint protegido rechaza sin X-Token
 - ✓ endpoint protegido rechaza sin JWT válido
 - ✓ PermissionChecker bloquea a un rol sin el permiso requerido
 - ✓ matching_niubiz no reutiliza una operación ya usada
 - ✓ matching_niubiz es incremental (no reprocesa pagos ya matcheados)
 - ✓ cancellation_book/confirm solo marca ids pendientes y reporta los que no aplica
 - ✓ sales_book invierte montos en notas de crédito
 - ✓ sales_book anula montos en documentos cancelados
-

18. Restricciones para el desarrollo

1. No romper la lógica de conciliación existente (`matching_niubiz`) sin cubrir con tests antes/después.
 2. No commitear datos reales de clientes ni reportes generados.
 3. Mantener el doble esquema de seguridad (X-Token + JWT + permisos) también desde el frontend.
 4. No introducir una librería de componentes UI pesada sin antes validarlo con el equipo — mantener Tailwind CSS como base.
 5. Mantener TypeScript estricto en el frontend; evitar `any` salvo justificación.
 6. Las reglas de autorización siempre en backend; el frontend solo oculta/muestra UI según permisos, nunca depende de eso para seguridad real.
 7. Cualquier nuevo endpoint de listados debe soportar paginación y filtros por fecha, pensando en que la UI los consumirá en tablas.
-

19. Instrucciones para el agente de desarrollo

Antes de modificar código:

1. Revisar esta ficha y contrastarla contra el estado real del repositorio (modelos, routers, schemas).
2. Priorizar el saneamiento del backend (Bloque 1) antes de construir el frontend.
3. Al iniciar el frontend, confirmar con el usuario decisiones de librería de componentes UI (o mantener solo Tailwind) antes de instalar dependencias adicionales.
4. Todo nuevo endpoint debe respetar el patrón ya existente: router en `routers/`, schema Pydantic en `schemas/`, modelo en `models/`, sesión vía `Depends(get_db)`.
5. Ejecutar y ampliar tests después de cada bloque una vez exista suite de pruebas.

Orden sugerido:

```
Saneamiento backend
↓
CRUD de soporte (usuarios, roles, permisos, listados paginados)
↓
Base del frontend Angular
↓
Módulos administrativos (usuarios, roles, dashboard)
↓
Flujo de conciliación (Niubiz + libro de cancelación)
↓
Reportes (libro de ventas, asiento contable)
```

20. Principio principal del proyecto

La prioridad absoluta es que este flujo funcione de punta a punta, ahora también desde una interfaz web:

SINCRONIZAR DATOS DE BSALE → CONCILIAR CON NIUBIZ → CONFIRMAR MANUALMENTE → GENERAR REPORTES CONTABLES

Todo desarrollo que no contribuya directamente a este flujo (incluyendo mejoras visuales del frontend) debe considerarse secundario mientras el panel no cubra el flujo completo.

21. Evolución futura

Panel de conciliación (MVP)
↓
Soporte para más pasarelas de pago además de Niubiz
↓
Conciliación automática vía API de Niubiz (sin carga manual de Excel)
↓
Dashboard financiero con indicadores históricos
↓
Integración directa con SIIGO (en vez de exportar Excel)
↓
Auditoría / historial de cambios por usuario
↓
Notificaciones (pagos sin conciliar, periodos pendientes de cierre)

La arquitectura actual (FastAPI + SQLAlchemy + MySQL, separación por routers/modelos/schemas) permite esta evolución sin necesidad de reescribir el backend existente.